

Lecture 3 Machine-Level Programming

Intel x86 Evolution: Milestones

- 8086: First 16-bit processor
- 386: First 32-bit processor
- Pentium 4E: First 64-bit processor
- Core i7: Multi-core machines
- ⋮

Advanced Micro Devices (AMD) is another main product that are compatible with Intel processors. A little bit slower but a lot cheaper.

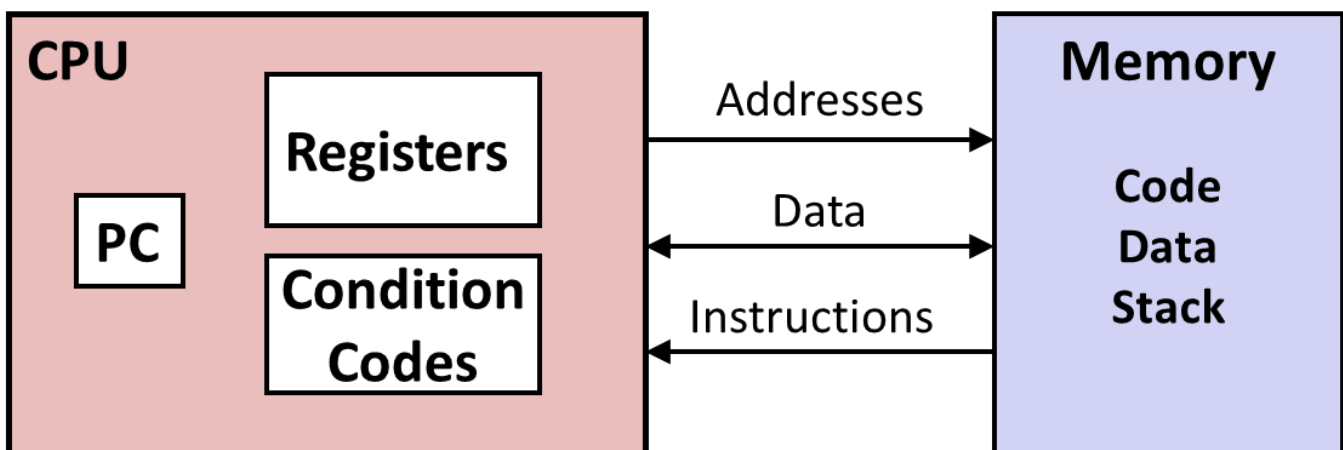
Machine Code Definition

Two of abstract models that are useful for computer systems are especially important for machine-level programming.

- Architecture(also **ISA**: instruction set architecture)
The format and behavior of a machine-level program is defined by ISA.(like ISA 32, x86-64)
It is also the parts that needs to understand to **write assembly code** in processor design.
- Memory model: A very large byte array

Assembly Programmer's View

Assembly-code representation transformed by compiler is very closed to machine code.



- PC: program counter. Store **address of next instruction**
- Registers: Heavily used programed data

- Condition codes: Store status information about most recent arithmetic operation. Used for conditional branching.
- Memory: Byte addressable array. Store code and user data. And a stack to support procedures.

Assembly Characteristics

Data Types

C declaration	Intel data type	Assembly-code suffix	Size (bytes)
char	Byte	b	1
short	Word	w	2
int	Double word	l	4
long	Quad word	q	8
char *	Quad word	q	8
float	Single precision	s	4
double	Double precision	l	8

Instructions

Most heavily used instructions include data movement instructions: MOV class

Instruction		Effect	Description
MOV	S, D	$D \leftarrow S$	Move
movb			Move byte
movw			Move word
movl			Move double word
movq			Move quad word
movabsq	I, R	$R \leftarrow I$	Move absolute quad word

x86-64 Integer Registers

x86-64 CPUs contains a set of **16 general-purpose registers** storing 64-bit values. It can store **integer** data or address. (Note that floating numbers has specific registers rather than those general registers)

general purpose	%rax	%eax	accumulate	%r8	%r8d
	%rbx	%ebx	base	%r9	%r9d
	%rcx	%ecx	counter	%r10	%r10d
	%rdx	%edx	data	%r11	%r11d
	%rsi	%esi	source index	%r12	%r12d
	%rdi	%edi	destination index	%r13	%r13d
	%rsp	%esp	stack pointer	%r14	%r14d
	%rbp	%ebp	base pointer	%r15	%r15d

The register can reference **low-order** 4 bytes.(compatible 32-bit architecture)

Moving Data

Moving data:

```
movq Source, Dest
```

Operand Types:

- Immediate: **Constant integer data**. (`$0x400` , `$-533`), encoded with 1, 2, 4, or 8 bytes
- Register: One of 16 integer registers(`%rax` , `%rdx`) But `%rsp` reserved for special use.
- Memory: 8 consecutive bytes of memory **at address given by register**. (`(%rax)` , `(%rdx)`)

Simple Memory Addressing Modes

Most general form:

$$D(Rb, Ri, S) \rightarrow \text{Mem}[\text{Reg}[Rb] + S * \text{Reg}[Ri] + D]$$

D: constant displacement

Rb: base register, any of 16 integer registers

Ri: index register, any except for `%rsp`
S: Scale, 1, 2, 4, or 8

For example,

```
%rdx:0xf000
%rcx:0x0100
```

Then

```
0x8(%rbx) -> 0xf000 + 0x8 = 0xf008
(%rdx,%rcx,4) -> 0xf000 + 4*0x0100 = 0xf400
0x80(,%rdx,2) -> 2*0xf000 + 0x80 = 0x1e080
```

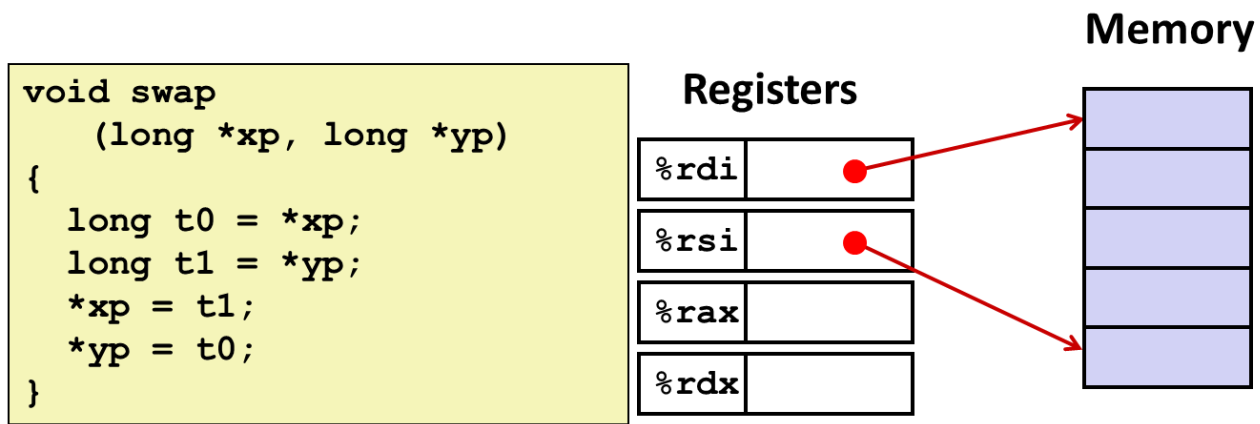
movq Operand Combination

	Source	Dest	Src, Dest	C Analog
movq	Imm	Reg	movq \$0x4,%rax	temp = 0x4;
		Mem	movq \$-147, (%rax)	*p = -147;
	Reg	Reg	movq %rax,%rdx	temp2 = temp1;
		Mem	movq %rax, (%rdx)	*p = temp;
	Mem	Reg	movq (%rax), %rdx	temp = *p;

Note that **no memory-memory transformation** with a single instruction.

Example: Understanding `Swap()`

memory to register, register to memory.



Register	Value
%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1

```
swap:
    movq    (%rdi), %rax    # t0 = *xp
    movq    (%rsi), %rdx    # t1 = *yp
    movq    %rdx, (%rdi)    # *xp = t1
    movq    %rax, (%rsi)    # *yp = t0
    ret
```

Some Arithmetic Operations

Two operand instructions:

```
addq,subq,imulq,salq,sarq,shrq,xorq,andq,orq
```

where `sarq` is arithmetic shift, `shrq` is logical shift. These instructions are all as following format:

```
Xq Src, Dest
```

And result is `Dest = Dest X Src`.

One operand instructions:

```
incq,decq,negq,notq
```

Processor State (x86-64 Partial)

In addition to integer registers, CPU maintains a set of **single-bit condition code** registers **describing attributes** of most recent arithmetic or logical operations.

Information about program:

- temporary data: `(%rax, ...)`
- location of runtime stack: `(%rsp)`
- status of recent tests: `(CF, ZF, SF, OF)`

Condition Codes (Implicit Setting)

- CF: carry flag(for unsigned)
- ZF: zero flag
- SF: sign flag(for signed)
- OF: overflow flag(for signed)

There are **implicitly set** by arithmetic operations, i.e., it is **as side effect**.

For example:

Addition:

- CF: if carry/borrow out from most significant bit(unsigned overflow)
- ZF: if the result is zero
- SF: if the result is negative
- OF: if signed overflow

It is **not set by** `leaq` **instruction**.

Condition Codes (Explicit Setting: Compare, Test)

```
cmpq Src2, Src1
```

It is like computing $a - b$ without destination

- CF: If carry/borrow from most significant bit
- ZF: if $a == b$.
- SF: if $(a - b) < 0$ (as signed)
- OF: if two's-complement (signed) overflow.

```
testq Src2, Src1
```

It is like computing $a \& b$ without destination.

- ZF: if $a \& b == 0$

- SF: if $a \& b < 0$

often use `testq %rax, %rax` to set ZF to be 0.

Reading Condition Codes

- SetX instructions

■ SetX Instructions

- Set low-order byte of destination to 0 or 1 based on combinations of condition codes
- Does not alter remaining 7 bytes

SetX	Condition	Description
<code>sete</code>	ZF	Equal / Zero
<code>setne</code>	$\sim$ ZF	Not Equal / Not Zero
<code>sets</code>	SF	Negative
<code>setns</code>	$\sim$ SF	Nonnegative
<code>setg</code>	$\sim (SF \wedge OF) \& \sim ZF$	Greater (Signed)
<code>setge</code>	$\sim (SF \wedge OF)$	Greater or Equal (Signed)
<code>setl</code>	$(SF \wedge OF)$	Less (Signed)
<code>setle</code>	$(SF \wedge OF) \mid ZF$	Less or Equal (Signed)
<code>seta</code>	$\sim CF \& \sim ZF$	Above (unsigned)
<code>setb</code>	CF	Below (unsigned)

`seta` is unsigned greater and `setb` is unsigned less.

It is only set **single byte** based on combination of condition and **does not alter remaining bytes**.

Typically use `movzbl` to **set upper bits to 0** to finish the job. (move zero byte to long(four bytes))

```
cmpq %rsi, %rdi # compare x:y
setg %al        # Set when >
movzbl %al, %eax # Zero rest of %rax
ret
```

When use `movzbl` operate low 32 bits, the high 32 bits are set as 0.

It is a performance of compare whether $x > y$.

Jumping

For jump, there exist **jump targets**, which is the addresses of the destination instructions. If jump target is encoded **as part of the instruction**, it is a **direct jump**(use a label in assembly code). If jump target is read from a register or a memory location, it is **indirect jump**(use `*` followed by an operand specifier).

jX instructions:

■ jX Instructions

- Jump to different part of code depending on condition codes

jX	Condition	Description
<code>jmp</code>	<code>1</code>	Unconditional
<code>je</code>	<code>ZF</code>	Equal / Zero
<code>jne</code>	<code>~ZF</code>	Not Equal / Not Zero
<code>js</code>	<code>SF</code>	Negative
<code>jns</code>	<code>~SF</code>	Nonnegative
<code>jg</code>	<code>~(SF^OF) & ~ZF</code>	Greater (Signed)
<code>jge</code>	<code>~(SF^OF)</code>	Greater or Equal (Signed)
<code>jl</code>	<code>(SF^OF)</code>	Less (Signed)
<code>jle</code>	<code>(SF^OF) ZF</code>	Less or Equal (Signed)
<code>ja</code>	<code>~CF & ~ZF</code>	Above (unsigned)
<code>jb</code>	<code>CF</code>	Below (unsigned)

C also allows `goto` statement. Jump to position designated by label. So the loop `for` , `while` , `do while` can all be performed by `goto` .

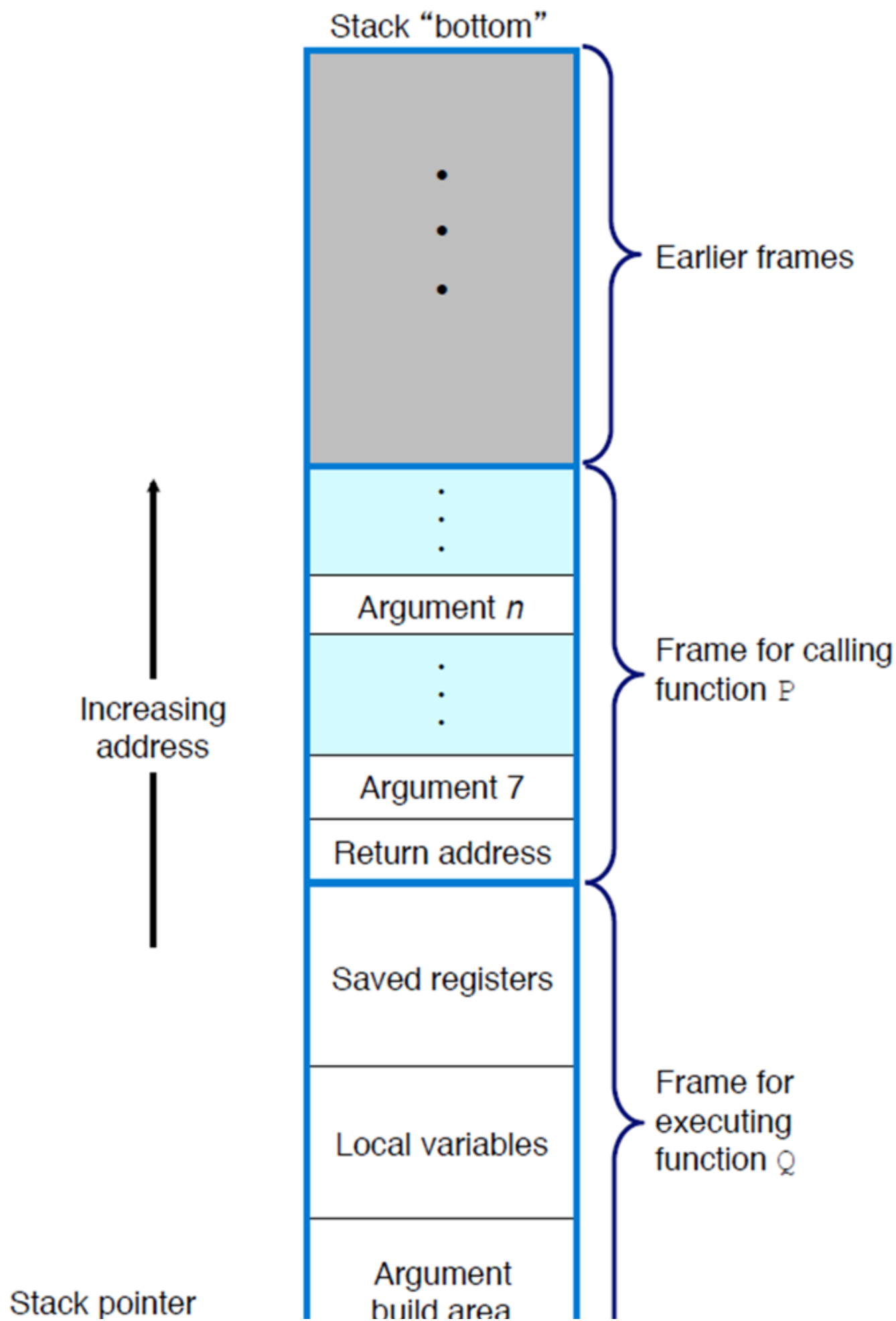
Procedures

A procedure call involves **passing both data(in the form of procedure parameters and return values)** and **control from one part of a program to another**. And it must **allocate space for local variables on entry** and **deallocate them on exit**.

Stack

X86-64 programs make use of the **program stack** to **support procedure calls**. The portion of the stack allocated for a single procedure call is called a **stack frame**. The top most stack frame is **delimited by the pointer** with `%rsp` **serving as the stack pointer**.

Stack Frame Structure





Suppose procedure P calls procedure Q. When P calls Q, the return address in P is pushed onto the stack, forming the end of P's stack frame to resume execution.

Q also uses the stack for local variables that cannot be stored in registers(not enough registers, need be accessed by array or structure references or must need an address)

The stack grows toward lower addresses. `%rsp` points to the top element of the stack. Data in stack can be manipulated by `pushq` and `popq`.

Transferring Control

Like jumps, call can either be direct(as a label) or indirect(given by `*`).

`call` pushes a return address on the stack and jump to the start of the called procedure. `ret` pops an address off the stack and jumps to this location.

Register Saving Conventions

- Caller Saved
Caller must save values in its stack frame before call
- Callee Saved
 - Callee saves values in its frame before using.
 - Callee restores values before returning.

x86-64 Linux Register Usage

Caller-saved
Return value

%rax

%rdi

%rsi

%rdx

%rcx

%r8

%r9

Caller-saved
Arguments

%r10

Caller-saved
Temporaries

%r11

**Callee-saved
Temporaries**

Special

%rbx

%r12

%r13

%r14

%r15

%rbp

%rsp

- **%rax**
 - return value
 - also caller-saved
 - can be modified by procedure
- **%rdi, ..., %r9**
 - Integer arguments
 - also caller-saved
 - Can be modified by procedure
- **%r10, %r11**
 - caller-saved
 - can be modified by procedure
- **%rbx, %r12, ..., %r15**
 - callee-saved
 - callee must save and restore
- **%rbp**
 - callee-saved
 - callee must save and restore

- may be used as frame pointer.
- compiler decides use of `%rbp`
- `%rsp`
 - special form of callee save
 - restored to original value upon exit from procedure

Classwork

Problem 3.8

Array Allocation

Contiguously allocated region of $L * N$ bytes where L is the size of one element.

Know the pointer arithmetic by unary operation `&` (get address) and `*` (access address)

For a pointer `p`, the expression `p+i` has value $x_p + L * i$ where x_p is the value of `p`.

Nested Arrays

The general principle of array allocation and referencing hold.

```
int A[5][3];

typedef int row3_t[3];
row3_t A[5];
```

the former two definition are equivalent.

Structure

All components of a structure are stored in a contiguous region of memory and a pointer to a structure is the address of the first byte.

Aligned data:

- Initial address of a variable with K -byte primitive data type must be multiple of K . For example, `int i[2]` will be allocated at `p+4, p+8` and so on.
- Total size of structure must be multiple of largest K , if not, padding bytes at the tail. compiler will insert gaps in structure to ensure correct alignment.

Reason:

memory access by chunks (4 or 8 bytes). If aligned, memory will not access a chunk with mix data type and need one more access.

Classwork

Problem 3.45

```
int silly(int n, int *p)
{
    int val, val2;

    if (n > 0)
        val2 = silly(n << 1, &val);
    else
        val = val2 = 0;

    *p = val + val2 + n;

    return val + val2;
}
```

```
silly:
    pushl %ebp
    movl %esp, %ebp
    subl $20, %esp
    pushl %ebx
    movl 8(%ebp), %ebx
    testl %ebx, %ebx
    jle .L3
    addl $-8, %esp
    leal -4(%ebp), %eax
    pushl %eax
    leal (%ebx, %ebx), %eax
    pushl %eax
    call silly
    jmp .L4
    .p2align 4,,7
.L3:
    xorl %eax, %eax
    movl %eax, -4(%ebp)
.L4:
    movl -4(%ebp), %edx
    addl %eax, %edx
    movl 12(%ebp), %eax
    addl %edx, %ebx
    movl %ebx, (%eax)
    movl -24(%ebp), %ebx
    movl %edx, %eax
    movl %ebp, %esp
    popl %ebp
    ret
```

This yields the following machine code:

Is the variable `val` stored on the stack? If so, at what byte offset (relative to `%ebp`) is it stored, and why is it necessary to store it on the stack?

Is the variable `val2` stored on the stack? If so, at what byte offset (relative to `%ebp`) is it stored, and why is it necessary to store it on the stack?

What (if anything) is stored at `-24(%ebp)`? If something is stored there, why is it necessary to store it?

What (if anything) is stored at `-8(%ebp)`? If something is stored there, why is it necessary to store it?

1. Is the variable `val` stored on the stack? If so, at what byte offset (relative to `%ebp`) is it stored, and why is it necessary to store it on the stack?

- **Yes**, `val` is stored on the stack.
- It is stored at byte offset `-4` relative to `%ebp` (i.e., at address `-4(%ebp)`).
- It is necessary to store `val` on the stack because its address is passed to the recursive call (`&val`), requiring `val` to have a fixed memory address.

2. Is the variable `val2` stored on the stack? If so, at what byte offset (relative to `%ebp`) is it stored, and why is it necessary to store it on the stack?

- **No**, `val2` is not stored on the stack.
- It is held in the register `%eax` (as the return value from the recursive call or set to 0 in the base case).
- It is not necessary to store `val2` on the stack because its address is never taken, so it can be kept in a register.

3. What (if anything) is stored at `-24(%ebp)` ? If something is stored there, why is it necessary to store it?

- The value of the callee-saved register `%ebx` is stored at `-24(%ebp)` .
- This is necessary because the function uses `%ebx` and must preserve its original value for the caller, as per the calling convention. The saved value is restored before returning.

4. What (if anything) is stored at `-8(%ebp)` ? If something is stored there, why is it necessary to store it?

- **Nothing** is stored at `-8(%ebp)` in this function.
- The code does not show any explicit store to or load from this location. The instruction `addl $-8, %esp` adjusts the stack pointer but does not use `-8(%ebp)` for data storage.